Array element initialization via pattern expansion

Document #:	P3110R0
Date:	2024-02-04
Project:	Programming Language C++
Audience:	Evolution

Reply-to:	James Touton <bekenn@gmail.com>

Contents

§ 1 Introduction

This paper introduces array element initializer patterns, which allow for the initialization of array elements using pattern expansion similar to pack expansion. This is useful when initializing an array of non-default-constructible type `T`, or when non-default initialization is desired and explicit array initialization syntax is too cumbersome or impossible (such as in a generic context, where the length of the array may depend on a template argument).

Example:

```
int a[57] = { 5 ... }; // initializes every element of a as 5
```

§ 2 Motivation and Scope

Initialization of array elements presently requires explicit syntax for each element when the desired initialization is more complicated than default or value initialization. If the array is large, this requirement becomes burdensome to the point that the developer may need to turn to alternative data structures that allow for initialization of new elements in a separate step after the data structure itself has already been initialized:

```
class E
{
public:
    E(int);
};

E a[100];       // error: E is not default constructible
E b[100] = { }; // error: E is not default constructible
E c[100] = { 0, 0, 0, /* 97 more zeros... */ }; // OK, but burdensome

template <size_t N>
void f()
{
    // error unless N matches the number of element initializers
    E x[N] = { 0, 0, 0, /* how many zeros here? */ };
}
```

As an aggregate, `std::array` suffers from the same limitations as built-in arrays, making it unwieldy to use with non-default-constructible types.

§ 3 Design Decisions

§ 3.1 General Approach

This feature is intended to follow the same general rules and syntax as pack expansion. Each is an example of pattern expansion, but whereas pack expansion is governed by the size of a parameter pack, expansion of array initializer patterns is governed by the size of the target array.

§ 3.2 Syntax

initializer-list:
    initializer-clause . . .$_{opt}$
    initializer-list , *initializer-clause* . . .$_{opt}$

The syntax for initializer lists does not change. Whereas an ellipsis was previously only permitted when an *initializer-clause* contained the name of an unexpanded parameter pack, an ellipsis may now also appear after the final *intializer-clause* to designate that *initializer-clause* as an array element initializer pattern. (If the *initializer-clause* contains the name of an unexpanded parameter pack, the meaning does not change; this is still a pack expansion rather than an array element initializer pattern.)

If the initializer pattern is (or ends with) a numeric literal, the ellipsis must be preceded by whitespace. This is because the characters making up the ellipsis would otherwise be interpreted as part of the numeric literal, even though the resulting literal would be invalid.

Example:

```
int a[27] = { 5... };   // error, 5... interpreted as an invalid numeric
        literal
int b[27] = { 5 ... };  // OK
int c[27] = { (5)... }; // OK, pattern does not end with a numeric literal
```

§ 3.3 Pattern Expansion

An array element initializer pattern may appear only as the final *initializer-clause* in an *initializer-list* that initializes an array of known bound. This works for both braced *initializer-list*s and parenthesized *expression-list*s. The pattern is replicated as many times as is necessary to explicitly initialize each element of the array.

It is permitted for an initializer list to contain both ordinary initializers and a terminating initializer pattern:

```
// This creates an array starting with the elements 1, 2, 3, 4 and ending
        with
// 96 repititions of the value 5:
int a[100] = { 1, 2, 3, 4, 5 ... };
```

Each initializer produced from a pattern is evaluated separately, exactly as if all initializers had been written out explicitly. There are no new restrictions on the content of an initializer clause.

For example, the pattern may contain a function call, which will be evaluated separately for every element:

```
#include <cassert>
#include <cstddef>

int array_elem(std::size_t index);

void f()
{
    std::size_t n = 0;
    int a[32] = { array_elem(n++)... };
    assert(n == 32);
}
```

Since the expanded initializers are evaluated left-to-right for both braced and parenthesized initialization, side effects are evaluated in order of increasing element index:

```
#include <generator>

std::generator<int> digits_of_pi();
```

```
void f()
{
    auto g = digits_of_pi();
    auto i = g.begin();
    int pi_100[100](*i++...);
}
```

§ 3.4 Brace Elision

The rules for aggregate initialization (9.4.2 [dcl.init.aggr]) permit inner braces to be elided when initializing members of a subaggregate. All known implementations of `std::array` take advantage of this feature to meet the standard's requirement that a `std::array` "can be list-initialized with up to `N` elements whose types are convertible to `T`" (24.3.7.1 [array.overview]). The implementation strategy (which is not mandated or even suggested by the standard, but which appears to be the only approach feasible without invoking specialized compiler-specific behavior) is to use a built-in array as the sole data member of `std::array`:

```
namespace std {
  template<class T, size_t N>
  struct array {
    /* non-data members */

    T _Elems[N];
  };
}
```

With brace elision, users may initialize a `std::array` with the same form of braced initializer as can be used to initialize a built-in array:

```
std::array<int, 5> x = { 1, 2, 3, 4, 5 };
```

…which, given the implementation strategy for `std::array`, is equivalent to the fully-braced form:

```
std::array<int, 5> x = { { 1, 2, 3, 4, 5 } };
```

The fully-braced form is not sanctioned by the standard.[1]

In principle, `std::array` could instead be implemented using a compiler-specific extension to make initialization more consistent with a built-in array, but it is clear that this feature must work for existing implementations of `std::array`.

To meet that need, an array element initializer pattern is permitted to appear in a braced initializer list wherein all initializers in the list appertain to elements of the same array:

```
struct A
{
    int a, b, c[20];
};

A a1 = { 1, 2, { 3, 4, 5 ... } };   // OK
```

```
A a2 = { 1, 2, 3, 4, 5 ... };        // error: some initializers appertain
        to
                                     // non-array elements

struct B
{
    int x[20];
};

B b1 = { { 1, 2, 3, 4, 5 ... } };    // OK
B b2 = { 1, 2, 3, 4, 5 ... };        // OK, all initializers appertain to
                                     // elements of the same array

struct C
{
    int a[20], b, c;
};

C c1 = { { 1, 2, 3 ... }, 4, 5 };    // OK
C c2 = { 1, 2, 3 ..., 4, 5 };         // error: initializer pattern cannot
        appear
                                        // in the middle of an initializer
        list
C c3 = { 1, 2, 3 ... };              // OK, b and c initialized to 0

struct D
{
    int a, b;
};

D d1[5] = { { 1, 2 }, { 3, 4 }... };     // OK
D  d2[5] = { 1, 2, 3, 4 ... };           // error: initializers do not
        appertain to
                                     // elements of an array
D d3[5] = { 1, 2, { 3, 4 }... };     // error: some initializers do not
                                     // appertain to elements of an array
```

§ 3.5 Parenthesized Initialization

C++20 introduced the ability to initialize an aggregate using a parenthesized *expression-list*. Arrays are aggregates, so that means arrays can be initialized with parentheses instead of braces.[2] This form of initialization has no equivalent to brace elision, so the addition of initializer patterns requires no special considerations.

§ 4 Wording

[CWG2149] (currently unresolved) points out an inconsistency in the wording with respect to array lengths inferred from braced initializer lists in the presence of brace elision. [P3106R0] attempts to resolve this issue by reformulating the rules for brace elision. Since this feature intersects with brace elision, the wording changes shown here are presented relative to [N4971] as modified by [P3106R0], under the assumption that [P3106R0] will be accepted.

Modify §7.6.1.4 [expr.type.conv] paragraph 2:

> If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (7.6.3 [expr.cast]). Otherwise, if the type is *cv* `void` and the initializer is `()` or `{}` (after ~~pack~~ pattern expansion, if any), the expression is a prvalue of type `void` that performs no initialization. *[…]*

Modify §9.4.1 [dcl.init.general] paragraph 18:

> An *initializer-clause* followed by an ellipsis is a ~~pack~~ pattern expansion (13.7.4 [temp.variadic]).

Insert a new paragraph after §9.4.1 [dcl.init.general] paragraph 18:

> A pattern expansion that is not a pack expansion is permitted to appear as the final element in a parenthesized *expression-list* that is used to initialize an array of known bound. Instantiation of the pattern expansion results in zero or more instantiations of the pattern such that the total number of elements in the *expression-list* matches the array bound.

Insert a new paragraph after §9.4.2 [dcl.init.aggr] paragraph 14 (as modified by [P3106R0]):

> A pattern expansion that is not a pack expansion is permitted to appear as the final element in a brace-enclosed *initializer-list* if all other elements of the *initializer-list* appertain to elements of the same array `u`, which shall be an array of known bound, and if the pattern would also appertain to an element of `u` (disregarding the array bound). Instantiation of the pattern expansion results in zero or more instantiations of the pattern such that the total number of elements in the *initializer-list* matches the array bound of `u`.

Modify §9.12.1 [dcl.attr.grammar] paragraph 4:

> In an *attribute-list*, an ellipsis may appear only if that *attribute*'s specification permits it. An *attribute* followed by an ellipsis is a ~~pack~~ pattern expansion (13.7.4 [temp.variadic]). *[…]*

Modify §13.7.4 [temp.variadic] paragraph 5:

5 A ~~*pack*~~ *pattern expansion* consists of a *pattern* and an ellipsis, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. ~~Pack~~ Pattern expansions can occur in the following contexts:

[…]

Modify §13.7.4 [temp.variadic] paragraph 7:

A pattern expansion is a *pack expansion* if the pattern names one or more packs that are not expanded by a nested pattern expansion; such packs are called *unexpanded packs* in the pattern. A pack whose name appears within the pattern of a pack expansion is expanded by that pack expansion. An appearance of the name of a pack is only expanded by the innermost enclosing pack expansion. ~~The pattern of a pack expansion shall name one or more packs that are not expanded by a nested pack expansion; such packs are called *unexpanded packs* in the pattern.~~ All of the packs expanded by a pack expansion shall have the same number of ~~arguments specified~~ elements. An appearance of a name of a pack that is not expanded is ill-formed.

[*Example: […] — end example*]

Modify §13.7.4 [temp.variadic] paragraph 8:

The instantiation of a ~~pack~~ pattern expansion considers items $\texttt{E}_1$, $\texttt{E}_2$, . . . , $\texttt{E}_N$~~,~~; ~~where~~ for pack expansions, N is the number of elements in ~~the pack expansion parameters~~ each unexpanded pack in the pattern. Each $\texttt{E}_i$ is generated by instantiating the pattern and replacing each unexpanded pack ~~expansion parameter~~ in the pattern with ~~its~~ the i^{th} element of the pack. Such an element, in the context of the instantiation, is interpreted as follows:

[…]

When N is zero, the instantiation of a ~~pack~~ pattern expansion does not alter the syntactic interpretation of the enclosing construct, even in cases where omitting the ~~pack~~ pattern expansion entirely would otherwise be ill-formed or would result in an ambiguity in the grammar.

Modify §13.7.4 [temp.variadic] paragraph 14:

> The instantiation of any other ~~pack~~ pattern expansion produces a list of elements $E_1, E_2, \ldots, E_N$.
>
> [*Note:* The variety of list varies with the context: *expression-list*, *base-specifier-list*, *template-argument-list*, etc. — *end note*]
>
> When *N* is zero, the instantiation of the expansion produces an empty list.
>
> [*Example: […] — end example*]

Add a new paragraph at the end of §13.7.4 [temp.variadic]:

> If a pattern expansion that is not a pack expansion appears in a context that is not explicitly permitted, the program is ill-formed.

Modify §13.8.3.3 [temp.dep.expr] paragraph 6:

> A *braced-init-list* is type-dependent if any element is type-dependent or is a pack expansion, or if the final element is a pattern expansion and all elements of the *braced-init-list* appertain to the same array of known bound where the bound is dependent on a template parameter.

Add an entry to §15.11 [cpp.predefined] table 22 [cpp.predefined.ft]:

Macro name	Value
`__cpp_array_elem_pattern_init`	⟨YYYYMMDD⟩L

[Editor's note: The value of the macro is to be determined at the editor's discretion.]

5 References

[CWG2149] Vinny Romano. 2015-06-25. Brace elision and array length deduction.
https://wg21.link/cwg2149

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.
https://wg21.link/n4971

[P3106R0] James Touton. 2024-02-03. Clarifying rules for brace elision in aggregate initialization.
https://wg21.link/p3106r0

---

1. This didn't stop older compilers from recommending the use of the fully-braced form, which has led to an awkward situation where fully-braced initializers are

common in user code despite being non-conformant with the standard. Current compilers do not warn against use of the fully-braced form.↩

2. `std::array` is also an aggregate, but because the standard doesn't mandate an implementation strategy, there is no conforming way to initialize `std::array` with parentheses.↩